

Università degli Studi di Salerno

Report di Tirocinio

Machine Learning su Dispositivi Embedded e TinyNIDS

TinyML, Architetture Embedded e Rilevamento Intrusioni di Rete

Autore: Valiante Samuele

Matricola: 0512119125

Anno Accademico 2025/2026

Indice

1. Introduzione a TinyML

- 1.1 Vantaggi di TinyML
- 1.2 Edge AI e Sistemi Embedded

2. Machine Learning e Applicazioni su Piattaforme Embedded

- 2.1 Applicazioni di ML su Piattaforme Embedded
- 2.2 Machine Learning: Definizione
- 2.3 Deep Learning e le Reti Neurali Artificiali (ANN)
- 2.4 Tipologie di Reti Neurali
- 2.5 Metodi di Apprendimento

3. TinyML: Architettura e Caratteristiche

- 3.1 Definizione e Caratteristiche Tecniche
- 3.2 Evoluzione e Motivazione
- 3.3 Approcci: EdgeML e CloudML

4. Hardware per TinyML

- 4.1 Microprocessore (μ P)
- 4.2 Microcontrollore (MCU)
- 4.3 SoC (System on a Chip)
- 4.4 Sistemi Embedded
- 4.5 Edge Device
- 4.6 FPGA (Field Programmable Gate Array)
- 4.7 Acceleratori Hardware

5. Software per TinyML

- 5.1 Framework e Ambienti di Sviluppo
- 5.2 Framework Open Source per SoC
- 5.3 Orientamento del Progetto: ML-First vs HW-First

6. Progetto TinyML: Fasi di Sviluppo

- 6.1 Workflow di Sviluppo
- 6.2 TinyML Workflow: Dal Modello all'Hardware
- 6.3 Applicazioni TinyML con IML

7. TinyNIDS: Sistema di Rilevamento delle Intrusioni di Rete

- 7.1 Motivazione e Contesto
- 7.2 Architettura del Sistema
- 7.3 Preprocessing del Traffico di Rete
- 7.4 Feature Extraction: SWCC
- 7.5 Costruzione del Modello TinyCNN

8. Valutazione delle Prestazioni di TinyNIDS

- 8.1 Analisi Comparativa dei Risultati
- 8.2 Analisi dei Risultati

9. Progettazione del sistema: Hardware, modelli e ottimizzazione

- 9.1 Scelta dell'hardware IoT
- 9.2 Scelta del modello di Machine Learning da utilizzare su un ESP32
- 9.3 Tecniche di ottimizzazione TinyML

10. Transformers

- 10.1 Apprendimento da sequenza a sequenza e attenzione
- 10.2 Il Transformer

11. Implementazione TinyNIDS

- 11.1 Raccolta e Generazione Autonoma del Dataset
- 11.2 Pipeline di Preprocessing
- 11.3 Costruzione e Addestramento del Tiny Transformer
- 11.4 Ottimizzazione: Pruning e Weight Clustering
- 11.5 Conversione in TensorFlow Lite
- 11.6 Integrazione nel Firmware ESP32
- 11.7 Misura delle Prestazioni su Hardware

12. Conclusioni

1. Introduzione a TinyML

TinyML (Tiny Machine Learning) è una tecnologia appartenente alla famiglia dell'Edge AI che consente di sviluppare dispositivi capaci di raccogliere ed elaborare dati autonomamente, senza dover interagire con dispositivi o server esterni. Questa tecnologia sfrutta la potenza dell'intelligenza artificiale (AI) e dell'Edge Computing per eseguire algoritmi di AI direttamente ai margini della rete, senza ricorrere al cloud computing.

1.1 Vantaggi di TinyML

L'adozione di TinyML comporta una serie di vantaggi significativi rispetto ai tradizionali approcci centralizzati:

- **Elaborazione immediata:** i dati vengono elaborati direttamente sul dispositivo, senza necessità di contattare un server remoto. Ciò permette risposte in tempo reale.
- **Latenza ridotta:** si eliminano i problemi di latenza derivanti dalla comunicazione con il server, rendendo il sistema adatto a scenari in cui il tempo di risposta è critico.
- **Sicurezza e privacy:** i dati sensibili non vengono trasmessi attraverso la rete, riducendo drasticamente i rischi legati alla privacy e alla sicurezza delle informazioni.
- **Ottimizzazione della larghezza di banda:** eliminando la necessità di trasmettere dati attraverso la rete, si evitano problemi di congestione e si riduce il consumo di banda.
- **Scalabilità:** questo paradigma consente lo sviluppo di sistemi distribuiti, scalabili e decentralizzati, adatti a deployment su larga scala.

1.2 Edge AI e Sistemi Embedded

I sistemi Edge AI possono essere distribuiti su varie configurazioni hardware. Un esempio paradigmatico è quello dei sistemi Embedded (dispositivi IoT), nei quali viene implementato l'Embedded ML (apprendimento automatico integrato). Questi dispositivi sono in grado di eseguire autonomamente funzioni di apprendimento automatico, elaborando i dati localmente senza dipendere da infrastrutture cloud.

2. Machine Learning e Applicazioni su Piattaforme Embedded

2.1 Applicazioni di ML su Piattaforme Embedded

L'apprendimento automatico sta avanzando rapidamente poiché i suoi bisogni architetturali richiedono sempre maggiori potenze di calcolo. Parallelamente, tuttavia, si sta diffondendo anche nell'ambiente IoT, su dispositivi con capacità di calcolo e risorse limitate. È in questo contesto che emerge il TinyML: un modello di ML progettato specificamente per dispositivi Embedded con risorse vincolate.

2.2 Machine Learning: Definizione

L'apprendimento automatico (Machine Learning) è una branca dell'intelligenza artificiale che si basa su algoritmi capaci di elaborare dati di input per effettuare previsioni o prendere decisioni. A partire dai dati, questi algoritmi sono in grado di individuare pattern e relazioni, effettuando previsioni sugli eventi futuri.

Un esempio applicativo è quello di una macchina industriale soggetta a guasti frequenti: alimentando un modello di ML con dati sullo stato della macchina, il sistema apprende le connessioni logiche tra i parametri operativi e le anomalie, consentendo previsioni accurate (inference) sulle probabilità di guasto.

2.3 Deep Learning e le Reti Neurali Artificiali (ANN)

Tra i diversi approcci del Machine Learning, il Deep Learning (DL) è quello che si ispira al funzionamento (semplificato) del cervello umano. In questo paradigma si utilizza una rete di neuroni simulati, denominata Artificial Neural Network (ANN), che modella le relazioni tra ingressi e uscite attraverso strati sovrapposti di elaborazione.

In una ANN, ogni strato è formato da neuroni collegati a tutti i neuroni dello strato precedente. Ogni connessione è associata a un peso (weight), che viene modificato durante la fase di addestramento in base al contributo che quel collegamento apporta all'output finale. Ad ogni neurone viene inoltre associato un bias, ovvero una costante additiva che funge da regolatore per adattare la risposta del neurone a diverse esigenze.

2.4 Tipologie di Reti Neurali

Le principali architetture di reti neurali utilizzate in ambito TinyML e Deep Learning sono:

- **FNN - Feed Forward Neural Network:** rappresenta l'architettura più semplice, nella quale i dati fluiscono in una sola direzione, dagli strati di input verso quelli di output. Non sono presenti cicli o connessioni retroattive.
- **CNN - Convolutional Neural Network:** utilizza la topologia feed forward per elaborare dati spaziali e tridimensionali. Grazie ai layer convoluzionali, è particolarmente efficace per il rilevamento di pattern all'interno di immagini, video e segnali audio.
- **RNN - Recurrent Neural Network:** caratterizzata da interconnessioni ricorrenti che creano una forma di memoria interna. Questa proprietà le rende adatte all'elaborazione di dati sequenziali, come testi nel Natural Language Processing (NLP), riconoscimento vocale ed elaborazione video.

2.5 Metodi di Apprendimento

Una volta definita l'architettura del modello, è necessario addestrarlo attraverso uno dei seguenti metodi di apprendimento:

- **Apprendimento supervisionato:** si forniscono al modello dati di input già etichettati con il loro corretto output. Il modello impara a generalizzare la relazione input-output a partire dagli esempi forniti.
- **Apprendimento non supervisionato:** il modello deve autonomamente trovare strutture e pattern nascosti nei dati, senza conoscere a priori l'output atteso. Questo approccio è utilizzato, ad esempio, per il rilevamento di anomalie.
- **Apprendimento semi-supervisionato:** approccio ibrido che combina dati etichettati e non etichettati. Una parte del dataset dispone dell'output corretto (supervisionato) mentre il resto no.
- **Apprendimento per rinforzo:** non esiste una risposta univoca corretta, ma il modello apprende tramite un meccanismo di prova-errore, ricevendo un feedback (ricompensa o penalità) per ogni azione intrapresa. L'algoritmo si migliora iterativamente ogni volta che effettua una scelta corretta.

3. TinyML: Architettura e Caratteristiche

3.1 Definizione e Caratteristiche Tecniche

TinyML è una classe di sistemi di apprendimento automatico integrato che implementa modelli di AI su Microchip e Microcontrollori (MCU), con un consumo energetico estremamente ridotto (dell'ordine di circa 1 mW) e pochi KB di RAM. Nei moderni MCU basati su architettura ARM sono state integrate CPU a 32 bit.

Gli MCU su cui gira TinyML non dispongono di un sistema operativo (OS), operano su singolo thread e non utilizzano la localizzazione dinamica della memoria. Le loro batterie garantiscono un'autonomia molto prolungata.

Nonostante i vincoli hardware, su questi dispositivi è possibile eseguire algoritmi di ML anche complessi, come le Reti Neurali Profonde (DNN). Ciò consente di implementare applicazioni quali il riconoscimento di immagini, il rilevamento di eventi in tempo reale e molto altro.

3.2 Evoluzione e Motivazione

Con il progresso tecnologico, TinyML ha trovato la sua ragione d'essere nell'esigenza di risolvere i problemi della centralizzazione dei dati. L'idea è quella di decentralizzare l'elaborazione, rendendo i dispositivi Embedded capaci di calcolare ed elaborare autonomamente, avvicinando la logica di elaborazione alla sorgente dei dati stessi.

Nell'Edge Computing tradizionale il processo è inverso: i dati vengono raccolti in periferia ma elaborati da un sistema centralizzato. TinyML sposta invece l'elaborazione direttamente sul dispositivo edge.

3.3 Approcci: EdgeML e CloudML

Per affrontare le sfide dei sistemi AI distribuiti esistono due principali approcci:

- **EdgeML:** mira a migliorare l'efficienza energetica, la potenza di calcolo e la latenza direttamente sui dispositivi edge. In anni recenti, l'EdgeML si sta orientando verso il paradigma 'addestrare nel cloud, inferire sul dispositivo', portando i modelli pre-addestrati su Arduino e dispositivi IoT.
- **CloudML:** punta al miglioramento delle capacità di elaborazione in cloud, affrontando problematiche quali l'Over-The-Air (OTA) update dei modelli e l'apprendimento federato (Federated Learning), che consente di addestrare modelli su dati distribuiti mantenendo la privacy.

4. Hardware per TinyML

4.1 Microprocessore (μ P)

Il microprocessore è un circuito integrato realizzato su microchip che contiene tutte le componenti principali di una CPU (Unità di Elaborazione Centrale). Si occupa esclusivamente delle operazioni di calcolo e deve essere affiancato da componenti esterni per memoria, I/O, ecc.

4.2 Microcontrollore (MCU)

Un Microcontrollore (MCU) è un circuito integrato dotato di tutti i componenti necessari per il funzionamento autonomo, inclusi:

- Microprocessore (μ P)
- RAM (memoria volatile)
- ROM (memoria non volatile)
- Interfacce di I/O
- Timer e altri periferici interni

4.3 SoC (System on a Chip)

Un SoC è un sistema che integra un MCU completo con periferiche avanzate su un unico chip. Può includere GPU, moduli Wi-Fi, Bluetooth, acceleratori hardware e molto altro. Questa integrazione lo rende particolarmente adatto per applicazioni IoT e Edge AI che richiedono alta densità funzionale in spazi ridotti.

4.4 Sistemi Embedded

I sistemi Embedded sono piccoli sistemi informatici dedicati progettati per svolgere compiti specifici. Sono tipicamente costruiti attorno a un MCU (o μ C) e progettati per applicazioni elettroniche ad hoc. Non sono sistemi general purpose, ma ottimizzati per un compito preciso.

4.5 Edge Device

Gli Edge Device sono dispositivi posti al bordo della rete, spesso non permanentemente connessi a infrastrutture cloud. Raccolgono dati da sensori sul campo, li aggregano, filtrano ed elaborano localmente prima di inviarli a sistemi server. Possono variare da semplici sensori a sistemi più complessi con capacità di elaborazione avanzate.

4.6 FPGA (Field Programmable Gate Array)

Le FPGA sono dispositivi logici programmabili la cui funzionalità non è definita dal produttore in fabbrica, ma dall'utente finale tramite linguaggi di descrizione hardware (HDL). Grazie alla loro riprogrammabilità e all'alta parallelizzazione, sono molto utilizzate per lo sviluppo e il prototipaggio di acceleratori per TinyML, specialmente in ambito di anomaly detection.

4.7 Acceleratori Hardware

Gli acceleratori hardware (Neural Processing Unit - NPU) sono processori specializzati progettati per migliorare le prestazioni nelle operazioni tipiche del Machine Learning, scaricando dal microprocessore principale i calcoli più pesanti. Le principali tipologie sono:

- **GPU (Graphic Processing Unit):** si differenzia dalla CPU nella sua specializzazione per l'elaborazione massivamente parallela di immagini. Alcune GPU, come quelle prodotte da NVIDIA, sono state ottimizzate anche per applicazioni TinyML.
- **TPU (Tensor Processing Unit):** processore realizzato appositamente per operazioni di ML. Grazie alle sue unità parallele, è in grado di eseguire un elevato numero di calcoli tensoriali in modo efficiente. È particolarmente indicato per le CNN, mentre le GPU lo sono per le RNN.
- **VPU (Visual Processing Unit):** dedicato all'elaborazione di immagini e a modelli di visione artificiale. Può includere interfacce dirette con dispositivi di acquisizione immagini (come fotocamere).
- **NNP (Neural Network Processor):** famiglia di processori progettati specificamente per compiti di intelligenza artificiale, ottimizzati per l'inferenza di reti neurali.

5. Software per TinyML

5.1 Framework e Ambienti di Sviluppo

Per sviluppare e far funzionare software per dispositivi TinyML, è necessario disporre di framework, librerie, sistemi operativi real-time e ambienti di sviluppo dedicati. Questi strumenti vengono classificati in base al loro scopo e al livello di astrazione che offrono allo sviluppatore.

I framework all-in-one rappresentano la soluzione più completa: consentono di integrare in un unico ambiente tutto il necessario per sviluppare applicazioni TinyML, coprendo le fasi di addestramento, ottimizzazione e deployment dei modelli di ML sui MCU. I principali sono:

- **TensorFlow Lite for Microcontrollers:** è un port del framework TensorFlow di Google, progettato per eseguire modelli di Deep Learning su dispositivi Embedded con pochissimi KB di memoria. Supporta quantizzazione e pruning per ridurre le dimensioni del modello.
- **Embedded Learning Library (ELL):** libreria sviluppata da Microsoft che consente di sviluppare e deployare modelli di ML su dispositivi embedded con risorse molto limitate.
- **Edge Impulse:** framework completo che consente di raccogliere dati, caricarli su cloud, addestrare modelli e deployarli su dispositivi embedded. Dispone di blocchi ML precostruiti che possono essere addestrati con nuovi dati. L'utente ha la possibilità di testare il modello su un simulatore prima di distribuirlo su un dispositivo fisico.

5.2 Framework Open Source per SoC

Esistono numerosi framework open source per gestire i SoC nelle applicazioni di Machine Learning. Un esempio rilevante è il CFU Playground, un framework full-stack open source che consente la progettazione rapida di acceleratori per sistemi Embedded. Molti di questi framework si basano su piattaforme FPGA per lo sviluppo e il testing di modelli, permettendo di implementare applicazioni come l'anomaly detection su hardware riconfigurabile.

5.3 Orientamento del Progetto: ML-First vs HW-First

Nello sviluppo di soluzioni TinyML esistono due filosofie di progettazione:

- **Focus orientato al Modello di ML:** la piattaforma hardware è scelta a priori e si progetta il modello di ML per adattarsi alle sue caratteristiche, ottimizzando successivamente il modello per l'hardware disponibile.
- **Focus orientato all'Hardware:** si parte dall'analisi dei colli di bottiglia dell'hardware esistente e si cerca di migliorarlo, sostituirlo o affiancarlo con acceleratori e piattaforme più performanti. L'obiettivo è migliorare le prestazioni tramite acceleratori hardware.

6. Progetto TinyML: Fasi di Sviluppo

6.1 Workflow di Sviluppo

Il processo che permette lo sviluppo di un sistema TinyML completo si articola nei seguenti passaggi:

1) Definizione degli Obiettivi

In questa fase si definisce il problema da risolvere, si identificano i tipi di dati da raccogliere, si stabilisce quale strategia adottare e quali modelli di ML considerare. Una chiara definizione degli obiettivi è fondamentale per guidare tutte le fasi successive.

2) Raccolta Dati e Creazione del Dataset

I dati raccolti devono essere classificati e strutturati in un dataset appropriato. In alcuni casi i dati non sono strutturati e devono essere pre-elaborati prima di poter essere utilizzati per l'addestramento.

3) Progettazione dell'Architettura del Modello

In questa fase si scelgono e progettano i modelli di rete neurale più adatti al problema. Si può partire da architetture pre-esistenti da adattare, oppure progettare una rete neurale da zero in funzione dei requisiti specifici dell'applicazione.

4) Allenamento del Modello

L'addestramento è un processo iterativo in cui il modello viene esposto ai dati di training e i suoi parametri vengono aggiornati per minimizzare l'errore. In ogni iterazione vengono calcolati sistemi di base (baseline) e si ricerca il modo di migliorare continuamente le prestazioni. Il modello può incontrare due problemi principali:

- **Underfitting:** il modello non ha acquisito conoscenze sufficienti dai dati di training e quindi performa male sia sui dati di training che su quelli di test.
- **Overfitting:** il modello ha memorizzato i dati di training in modo troppo specifico e non è in grado di generalizzare su dati nuovi, rendendolo inutilizzabile in produzione.

5) Validazione del Modello

Il dataset viene suddiviso in tre parti per una corretta valutazione del modello:

- 60% per l'allenamento (training set)
- 20% per la convalida (validation set) - utilizzato per verificare che il modello non vada in overfitting durante il training
- 20% per il test (test set) - misura le prestazioni su dati completamente nuovi e mai visti

6.2 TinyML Workflow: Dal Modello all'Hardware

Una volta definito e validato il modello di ML, è necessario adattarlo per il deployment su sistemi embedded, tenendo conto delle limitazioni hardware specifiche del dispositivo target. Questo processo include la quantizzazione (riduzione della precisione numerica dei pesi), il pruning (eliminazione delle connessioni meno rilevanti) e la conversione del modello in formati compatibili con i framework embedded scelti.

6.3 Applicazioni TinyML con IML

Il modello di ML da solo non è sufficiente: è necessario costruire un'applicazione completa che interagisca con i dati di input, li pre-elabori, li passi al modello e gestisca l'output per attuare le decisioni sulla piattaforma hardware. Il flusso tipico di un'applicazione TinyML è il seguente:

- Acquisizione dei dati di input
- Pre-elaborazione dei dati per renderli compatibili con il modello
- Esecuzione dell'inferenza tramite l'interprete TensorFlow Lite
- Post-elaborazione dei risultati
- Attuazione delle decisioni sull'hardware (uscita dispositivo)

7. TinyNIDS: Sistema di Rilevamento delle Intrusioni di Rete

7.1 Motivazione e Contesto

Con l'evoluzione delle reti di comunicazione dal 5G al 6G, la sicurezza informatica è diventata sempre più critica a causa dell'enorme numero di dispositivi connessi e della natura decentralizzata delle architetture di rete. Le reti 6G promettono comunicazioni ultra-veloci, latenza minima e connettività massiva, abilitando applicazioni avanzate come la guida autonoma, la telemedicina da remoto, le smart city e la realtà virtuale immersiva.

Tuttavia, questa complessità introduce nuove vulnerabilità. Tra i principali rischi di sicurezza si annoverano gli attacchi DDoS (Distributed Denial of Service), che sfruttano l'enorme numero di dispositivi connessi per sovraccaricare le risorse di rete, e gli attacchi Man-in-the-Middle (MitM), che compromettono l'integrità delle comunicazioni a bassa latenza.

In questo contesto nasce TinyNIDS: un sistema CNN-based per il rilevamento delle intrusioni di rete, progettato per il deployment su modelli TinyML in ambienti 6G.

7.2 Architettura del Sistema

L'architettura di TinyNIDS comprende diversi moduli che lavorano insieme per abilitare il rilevamento delle intrusioni in tempo reale, in modo decentralizzato ed efficiente su dispositivi edge:

- **Data Acquisition Module:** raccoglie dati di traffico di rete da dispositivi IoT, endpoint mobili e router edge. I dati includono indirizzi IP, tipi di protocollo, dimensioni dei pacchetti e timestamp.
- **Preprocessing Module:** converte i dati simbolici in rappresentazioni numeriche tramite tecniche come il quad-splitting per gli indirizzi IP e il one-hot encoding per i tipi di protocollo. Applica successivamente la normalizzazione min-max per garantire la compatibilità con i modelli TinyML.
- **Feature Extraction Module:** utilizza l'algoritmo SWCC (Similarity Weighted Cosine Correlation) per identificare le feature di traffico più rilevanti, riducendo la dimensionalità dei dati.
- **TinyCNN Intrusion Detection Module:** il cuore del sistema, responsabile della classificazione del traffico come normale o anomalo.
- **Alert and Response Module:** rileva il traffico anomalo e attiva le azioni di sicurezza appropriate (alert in tempo reale, blocco di IP malevoli, limitazione del traffico sospetto, attivazione di regole firewall).
- **Centralized Monitoring and Model Update Module:** consente la visualizzazione e il monitoraggio centralizzato da parte degli amministratori, con aggiornamenti periodici del modello via OTA (Over-The-Air).

7.3 Preprocessing del Traffico di Rete

Il traffico di rete mobile contiene frequentemente dati simbolici (indirizzi IP, tipi di protocollo) che non possono essere direttamente utilizzati nei modelli di ML. La fase di preprocessing li converte in rappresentazioni numeriche:

Conversione degli Indirizzi IP

Un indirizzo IP nel formato A.B.C.D viene suddiviso in quattro componenti indipendenti, ciascuno dei quali costituisce una feature numerica distinta. Il vettore trasformato risultante è: $x_IP = (A, B, C, D)$, con A, B, C, D appartenenti a $\{0, 1, \dots, 255\}$.

One-Hot Encoding per Dati non Gerarchici

Per dati simbolici privi di gerarchia intrinseca (come i tipi di protocollo), viene applicato l'one-hot encoding: ogni tipo di protocollo p_i è mappato a un vettore binario v_i di lunghezza n , dove solo la posizione i è uguale a 1 e tutte le altre sono 0.

Normalizzazione Min-Max

Per garantire che tutte le feature abbiano scale comparabili ed evitare squilibri che penalizzerebbero le prestazioni del modello, viene applicata la normalizzazione min-max: $x'_i = (x_i - x_{\min}) / (x_{\max} - x_{\min})$, con x'_i in $[0, 1]$.

7.4 Feature Extraction: SWCC

Dopo il preprocessing, il modulo di Feature Extraction estrae le feature più significative dal traffico di rete. Viene utilizzato l'algoritmo SWCC (Similarity Weighted Cosine Correlation):

1. Le feature estratte (dimensioni dei pacchetti, tipi di protocollo, pattern di traffico, indirizzi IP) vengono organizzate in una matrice X di dimensione $m \times n$, dove m è il numero di campioni e n il numero di feature.
2. La similarità coseno tra le coppie di feature viene calcolata per quantificare il grado di allineamento tra i vettori di feature.
3. Per ogni feature x_i viene calcolato il valore SWCC $S(x_i)$, che riflette il contributo aggregato della feature basato sulla sua similarità con le altre. Questo valore priorizza le feature ad alta rilevanza per il rilevamento delle intrusioni.

7.5 Costruzione del Modello TinyCNN

Il cuore di TinyNIDS è il modello TinyCNN, una CNN leggera ottimizzata per il deployment su dispositivi edge con risorse limitate. La sua architettura comprende:

Input Layer

Il traffico di rete viene rappresentato come una matrice bidimensionale X di dimensione $m \times n$, dove m è il numero di feature e n la lunghezza di ciascun vettore di feature. Questa rappresentazione permette ai layer convoluzionali di catturare le relazioni spaziali tra i dati.

Layer Convoluzionali

I layer convoluzionali utilizzano kernel (filtri) per estrarre pattern locali dalla matrice di input. L'operazione di convoluzione in posizione (i,j) genera una feature map F , calcolando la somma pesata dei valori della matrice di input moltiplicati per il kernel K , più il termine di bias b . Vengono impiegate convoluzioni depthwise separabili per ridurre la complessità computazionale, minimizzando il numero di parametri e accelerando l'inferenza.

Layer di Pooling

I layer di pooling riducono le dimensioni delle feature map, prevenendo l'overfitting e riducendo i costi computazionali. Il max pooling conserva il valore più significativo di ogni regione, preservando le feature importanti mentre riduce la dimensionalità dei dati.

Fully Connected Layer e Funzione di Attivazione

Dopo il pooling, le feature map vengono appiattite in un vettore monodimensionale $z = \text{Flatten}(P)$, che viene passato al fully connected layer. Per la classificazione binaria (traffico normale vs. anomalo), viene utilizzata la funzione di attivazione sigmoide: $\sigma(x) = 1/(1 + e^{-x})$, che mappa l'output nell'intervallo $[0,1]$, rappresentando la probabilità che il campione di input sia anomalo.

Funzione di Loss

Il modello viene addestrato utilizzando la funzione di cross-entropia binaria, che misura la differenza tra l'output predetto e l'etichetta reale. Il modello minimizza questa loss tramite algoritmi di ottimizzazione come Adam o SGD.

8. Valutazione delle Prestazioni di TinyNIDS

8.1 Analisi Comparativa dei Risultati

I risultati sperimentali di TinyNIDS dimostrano le prestazioni superiori del modello TinyCNN rispetto ai modelli tradizionali come CNN standard, RNN e reti neurali BP (Back Propagation), nelle medesime condizioni sperimentali. La tabella seguente riassume le principali metriche di performance:

Modello	Accuracy	Recall	F1-Score	FPR	FNR	Latency
TinyCNN	96.5%	94.2%	95.3%	2.1%	3.7%	12 ms
CNN	93.8%	91.0%	92.3%	3.5%	4.5%	25 ms
RNN	92.1%	89.4%	90.7%	5.2%	6.8%	35 ms
BP Neural Network	88.3%	85.0%	86.6%	7.8%	9.2%	40 ms

8.2 Analisi dei Risultati

TinyCNN consegue risultati superiori su tutte le metriche principali:

- Accuratezza:** 96.5% contro il 93.8% della CNN standard, il 92.1% della RNN e l'88.3% della rete BP.
- False Positive Rate (FPR):** TinyCNN registra il valore più basso (2.1%), minimizzando i falsi allarmi in modo più efficace rispetto a RNN (5.2%) e BP (7.8%).
- False Negative Rate (FNR):** il valore di 3.7% è competitivo, sebbene TinyCNN mostri prestazioni leggermente inferiori alla CNN standard in scenari complessi (tipi di intrusione 4 e 6), suggerendo margini di miglioramento.
- Latenza:** 12 ms, significativamente inferiore rispetto agli altri modelli (CNN: 25ms, RNN: 35ms, BP: 40ms), confermando la sua idoneità per scenari di edge computing in tempo reale.

Questi risultati dimostrano che TinyCNN raggiunge un ottimo equilibrio tra minimizzazione dei falsi positivi e dei falsi negativi, aspetto essenziale per il rilevamento delle intrusioni in tempo reale in ambienti 6G con risorse vincolate.

9. Progettazione del sistema: Hardware, modelli e ottimizzazione

L'idea per il nostro progetto è quella di creare un Intrusion Detection System (IDS) molto leggero, direttamente su una board IoT. Quindi dobbiamo scegliere la board che più si adatta alle nostre esigenze e adattargli un modello di ML che permetta di rilevare e prevenire, come già anticipato nella sezione 7.1, minacce come attacchi DDoS e Man-In-The-Middle. Ora ci viene il dubbio, quali schede IoT utilizzare? Quale modello più leggero possiamo metterci su? Con quali tecniche? Nelle prossime sezioni risponderemo a ciò.

9.1 Scelta dell'hardware IoT

I dispositivi che possiamo prendere in considerazione riguardo un progetto di TinyML sono a basso consumo e che richiedono pochissima RAM. I più di spicco che abbiamo sono:

- **ESP32:** è probabilmente la scelta perfetta per implementare un TinyNIDS poichè, avendo moduli wi-fi e bluetooth integrati ci permette di simulare un vero e proprio nodo di rete che ci consentirà di analizzare il traffico in tempo reale.
- **Arduino Nano 33 BLE Sense:** si interfaccia in modo nativo e perfetto con Edge Impulse e dispone di un MCU ARM Cortex-M4 che gestisce benissimo l'inferenza con reti neurali quantizzate in formato int8.
- **Raspberry Pi Pico:** è una scheda economica ma potente (dual-core) la quale la rende ottima per testare modelli leggeri in un ambiente senza sistema operativo

La scheda che si adatta di più alle nostre esigenze è la ESP32 a cui faremo riferimento per il resto del documento.

9.2 Scelta del modello di Machine Learning da utilizzare su un ESP32

Vediamo, una volta decisa quale scheda IoT utilizzare, quale modello di Machine Learning è più adatto per i nostri scopi. Partiamo dai più leggeri:

- **Alberi decisionali e Random Forests:** essi sono perfetti per i dati di rete strutturati. Sono modelli i quali una volta addestrati si traducono con una struttura di if-else molto leggera.
- **FNN:** essi sono molto più semplici rispetto alle CNN poichè la rete neurale si riduce di pochi livelli, uno in input, uno o due nascosti(16 o 32 neuroni) e uno di output i quali entrano perfettamente in un ESP32 e consente di classificare dati provenienti da sensori o pacchetti di rete.
- **TinyCNN:** è un architettura convoluzionale costruita su misura e semplificata (2 layer convoluzionali), qui i pacchetti di rete vengono tradotti in matrici bidimensionali per trovarne i pattern malevoli. Più pesante delle precedenti ma essendo semplificata va bene per un ESP32.
- **CNN:** è un architettura convoluzionale che processa dati in maniera molto precisa il che va bene ma per un dispositivo più potente del nostro ESP32. Quindi abbiamo bisogno di determinate tecniche di TinyML per poter far sì che questo modello sia eseguibile su questa scheda.

- **Tiny Transformers:** qui andiamo quasi all'estremo in cui eseguire un modello così pesante su una scheda da risorse limitate è quasi impossibile a causa del meccanismo della self-attention che consuma memoria in modo esponenziale rispetto all'input. Per farlo è complesso ma usando tecniche di TinyML può essere fattibile.

Analizzeremo queste tecniche di TinyML nel prossimo paragrafo.

9.3 Tecniche di ottimizzazione TinyML

Dopo aver scelto una scheda IoT come l'ESP32 che pur essendo potente non lo è abbastanza da implementare un architettura a transformers, sono state definite delle tecniche di TinyML per rendere fattibile quest'impresa. Di seguito le elenchiamo:

- **Quantizzazione:** solitamente una rete neurale dispone di pesi a 32 bit, un'idea potrebbe essere quella di abbassare questi numeri fino a interi a 4 bit o con la Binary Neural Network con pesi tra -1 e +1, come si sta sperimentando. Questo fa sì di risparmiare tante risorse di calcolo con però un'accuratezza leggermente inferiore.
- **Pruning (potatura):** quando la rete neurale viene addestrata vi sono alcune connessioni tra neuroni il cui peso è irrilevante poiché vicino allo 0 che alla fine non influenzeranno la decisione finale. Un'operazione utile che ci riduce di molto le operazioni matematiche necessarie è rimuovere queste connessioni inutili.
- **Weight Clustering (Condivisione dei pesi):** invece di avere migliaia di pesi tutti leggermente diversi, il modello viene forzato a raggruppare pesi simili in un numero limitato di "cluster" (ad esempio solo 16 o 64 valori unici in tutto il modello). Facendo così il modello deve memorizzare solo una piccola parte delle info, di conseguenza viene risparmiata molta memoria flash.
- **Knowledge distillation:** questa tecnica agisce sull'addestramento e consiste nel prendere un modello già addestrato più grande (teacher) e il nostro modello (student) viene addestrato per riprodurre il comportamento del modello teacher. Il risultato di ciò è un modello che ha quasi lo stesso comportamento del teacher ma con meno parametri.
- **Neural Architecture Search (TinyNAS):** qui invece di crearci a mano la nostra architettura sperando che sia delle dimensioni giuste, affidiamo il compito ad un algoritmo che ce ne genera molteplici, ognuna di grandezza diversa e le testa ognuna in base ai vincoli che l'hardware offre (nel nostro caso quelli dell'ESP32). Tra tutte le migliaia di architetture, l'algoritmo sputerà fuori quella perfetta per l'ESP32.
- **Approssimazione dell'Attenzione (Linear Attention):** dato che la Self-Attention standard ha una complessità $O(N^2)$. Se la sequenza raddoppia, la memoria richiesta quadruplica, saturando l'ESP32 all'istante. Nel TinyML, la si sostituisce con la Linear Attention, che usa trucchi matematici (come l'eliminazione della funzione Softmax o l'uso di kernel approssimati) per rendere il calcolo lineare $O(N)$.
- **Sostituzione dei layer Feed-Forward:** Nelle architetture a transformers vi è un blocco denominato Feed-Forward che si incarica di elaborare ulteriormente le informazioni dopo che gli è stata applicata l'attenzione. Dato che la nostra architettura è molto ridotta rispetto a quelle standard possiamo rimpiazzare questo blocco con uno che non faccia alcune operazioni che non ci interessano snellendo ancor di più la stessa.

10. Transformers

Avendo scelto di implementare per il nostro sistema un'architettura a transformers vedo utile dedicargli un capitolo per capirne com'è fatta e come funziona.

10.1 Apprendimento da sequenza a sequenza e attenzione

Il modello sequenza a sequenza detto anche Seq2Seq è un modello che, come dice il nome, prende una sequenza in input e la traduce in un'altra sequenza di output. Esso si compone di un **encoder(o LSTM)** che prende la sequenza e la mappa in un vettore n-dimensionale e un **decoder** che prende questa mappatura, la traduce e ne fornisce un output.

Prendiamo ad esempio due lingue, il nostro modello dovrà tradurre da italiano a inglese, l'encoder prende la frase in italiano (sequenza) e la mappa in una lingua immaginaria che conoscono sia esso che l'encoder. Dopodiché l'encoder prende la frase mappata nella lingua immaginaria e la riporta in inglese (sequenza).

Ovviamente i due componenti devono essere addestrati su come interpretare questa lingua immaginaria.

Prima di parlare di ciò voglio enunciare che questo modello utilizza un meccanismo di attenzione che, esaminando l'input ad ogni passo decide quali altre parti sono importanti.

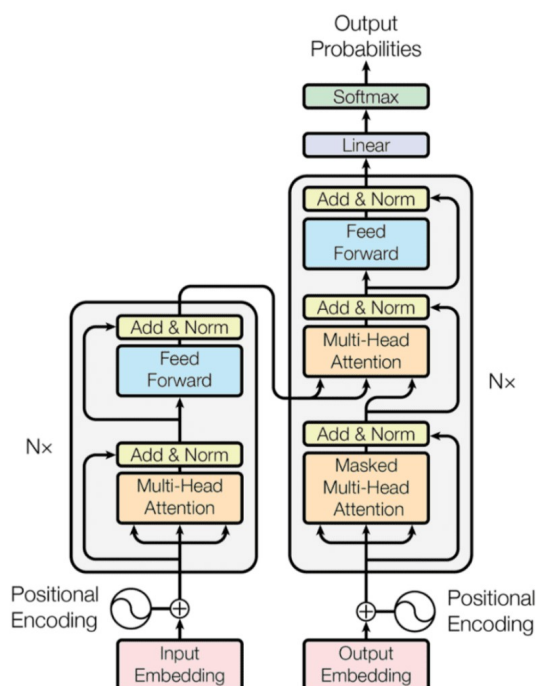
Praticamente nel nostro esempio, l'LSTM non dà al decoder solo la traduzione in lingua immaginaria della sequenza ma gli dà delle parole in più per aiutarlo nel capire meglio il contesto della frase. Grazie all'attention si occupa di capire quali di queste parole chiave sono importanti e gli attribuisce un peso che darà al decoder insieme alla sequenza codificata.

10.2 Il Transformer

Esso, basandosi sul meccanismo dell'attention abbandona totalmente l'uso di reti ricorrenti (RNN) rendendolo molto più efficiente ma più pesante dato che questo meccanismo consuma memoria in modo esponenziale rispetto l'input.

Come si può vedere dall'immagine accanto, a sinistra abbiamo l'encoder e a destra il decoder. Ovviamente questi componenti vengono impilati N volte sopra per rendere il modello più profondo e intelligente.

All'inizio le parole vengono valutate e dato che vengono processate senza ordine e se ne perde l'ordine ne viene assegnata una quantità matematica che lo identifica.



Il meccanismo di **self-attention** permette all'encoder inizialmente di prendere una parola e guardare le altre per capire il contesto della frase.

Nell'Encoder poi abbiamo il **Masked Multi-Head Attention** che permette durante l'addestramento di non svelare al modello parole future così che impari a dedurle da solo (apprendimento non supervisionato).

Add & Norm e **Feed Forward** sono blocchi di supporto per i dati estratti per l'apprendimento o per evitare che diventino troppo grandi e per elaborare le info estratte dall'attention.

Il blocco **Linear** trasforma i dati elaborati in un punteggio per ogni parola.

Il blocco **Softmax** trasforma questi punteggi in una probabilità da 0 a 1. La parola con la probabilità più alta è quella che viene scelta.

11. Implementazione TinyNIDS

Il seguente capitolo tratta di come andremmo ora ad implementare il nostro Intrusion detection system sul nostro esp32. Il tutto verrà svolto in varie fasi di sviluppo partendo dall'acquisizione del dataset con i dati per addestrare il modello, fino a poi l'implementazione del modello e del suo relativo addestramento. Nelle fasi finali il modello addestrato sarà ottimizzato (come già illustrato in sez. 9.3) e portato sull'esp32, infine saranno misurate le metriche reali (latenza, uso RAM/Flash, accuracy) e confrontate con le stime teoriche dei cap. 8-9, iterando sulle tecniche di ottimizzazione se i vincoli hardware non sono rispettati.

11.1 Raccolta e Generazione Autonoma del Dataset

A differenza dei riferimenti bibliografici citati nei capitoli precedenti, la validazione sperimentale del sistema TinyNIDS proposto in questo lavoro non si basa su un dataset pubblico preesistente, bensì su un dataset generato in modo autonomo attraverso un ambiente di laboratorio controllato, appositamente allestito per catturare traffico di rete realistico sia in condizioni operative normali sia durante attacchi informatici simulati.

Configurazione dell'ambiente di acquisizione

Il Data Acquisition Module è stato realizzato configurando la scheda ESP32 come Access Point WiFi indipendente (Soft-AP), creando una rete di laboratorio isolata a cui sono stati collegati due dispositivi client (un personal computer e uno smartphone).

In questa configurazione l'ESP32 opera come nodo centrale attraverso cui transita l'intero traffico di rete dei dispositivi connessi, condizione necessaria per l'osservazione completa dei pacchetti scambiati.

La cattura del traffico sfrutta la modalità promiscua (promiscuous mode) offerta nativamente dal chip WiFi dell'ESP32, tramite la quale il dispositivo intercetta ogni frame IEEE 802.11 che transita sulla propria interfaccia radio, indipendentemente dal fatto che ne sia o meno il destinatario diretto. Per limitare la cattura ai soli frame contenenti dati utili — escludendo frame di gestione e di controllo quali beacon, probe e ACK — è stato applicato

un filtro nativo del driver WiFi (WIFI_PROMIS_FILTER_MASK_DATA). Un ulteriore filtro, basato sul confronto con l'indirizzo MAC (BSSID) del proprio Access Point, è stato introdotto per scartare frame appartenenti a reti WiFi limitrofe eventualmente captate per la condivisione del medesimo canale radio, garantendo così l'integrità del dataset raccolto.

Estrazione delle feature

Per ciascun frame dati captato, il firmware dell'ESP32 esegue il parsing dell'header IEEE 802.11 (e dell'eventuale intestazione LLC/SNAP) per individuare l'inizio del pacchetto IP incapsulato, da cui vengono estratti in tempo reale i seguenti campi: indirizzo IP sorgente, indirizzo IP destinazione, numero di protocollo di livello trasporto (identificativo posizionato al byte 9 dell'header IPv4) e lunghezza complessiva del pacchetto. Per la cattura degli attacchi di tipo Man-in-the-Middle, il parsing è stato esteso al riconoscimento dei pacchetti ARP (identificati tramite il campo EtherType 0x0806), da cui vengono estratti l'indirizzo IP del mittente dichiarato, l'indirizzo IP del bersaglio e la tipologia di operazione (richiesta o risposta ARP). I dati estratti vengono trasmessi via interfaccia seriale (USB) verso un elaboratore host, dove uno script Python li acquisisce, associa a ciascun campione un timestamp in formato ISO 8601 e un'etichetta di classe (normale o attacco), per poi memorizzarli in formato CSV.

Generazione del traffico normale

Il traffico appartenente alla classe “normale” è stato ottenuto catturando l'attività di rete spontanea generata dai due dispositivi client durante l'invio da pc a smartphone di pacchetti ICMP Echo Request e da smartphone a pc di pacchetti ICMP Echo Reply tramite il comando ping mentre entrambi erano connessi all'ESP32. Per arricchire il dataset l'AP è stato configurato per ottenere l'accesso a internet e nel mentre avveniva la cattura sono state effettuate svariate operazioni di navigazione sul web.

Generazione del traffico di attacco

Il traffico appartenente alla classe “attacco” è stato generato simulando, in ambiente di laboratorio controllato, due tipologie di attacco tra quelle discusse nei capitoli precedenti:

- **DDoS (SYN Flood):** realizzato mediante lo strumento hping3, che ha generato un flusso ad alta frequenza di pacchetti TCP con il solo flag SYN attivo, senza mai completare il three-way handshake, verso l'indirizzo dell'Access Point. Il firmware è stato configurato per filtrare a bordo esclusivamente i pacchetti TCP con flag SYN attivo, individuato tramite l'analisi del byte 13 dell'header TCP.
- **MITM (ARP Spoofing):** realizzato mediante lo strumento arpspoof, che ha inviato messaggi ARP reply non richiesti e falsificati a entrambi gli endpoint della comunicazione (Access Point e client), inducendo ciascuno ad associare erroneamente l'indirizzo IP dell'altro all'indirizzo MAC del dispositivo attaccante, dirottando così il flusso di traffico attraverso quest'ultimo.

Composizione finale del dataset

Il dataset risultante è composto da un totale di 23302 campioni etichettati, distribuiti come riportato in questa tabella.

Classe	Sottocategoria	Campioni	Protocolli osservati
Normale	—	31401	ICMP, UDP, TCP, IGMP
Attacco	DDoS (SYN Flood)	17240	TCP (solo flag SYN)
Attacco	MITM (ARP Spoofing)	17749	ARP (17550 reply, 199 request)
Totale		66390	

Il rapporto tra le due classi principali (31.401 campioni normali contro 34.989 di attacco, circa 0,90:1) risulta pressoché bilanciato. Un residuo sbilanciamento a livello di finestre di addestramento (dopo la segmentazione in sequenze) viene comunque gestito mediante pesi di classe in fase di addestramento del modello.

11.2 Pipeline di Preprocessing

Il dataset grezzo, così come raccolto dal Data Acquisition Module, non è direttamente utilizzabile come input per il Tiny Transformer: contiene stringhe (indirizzi IP), valori categorici (protocollo) e timestamp assoluti, mentre una rete neurale richiede in ingresso un tensore di soli valori numerici, su scale tra loro comparabili.

La pipeline di preprocessing colma questo divario, trasformando ciascun pacchetto catturato in un vettore numerico a dimensione fissa e organizzando i pacchetti in sequenze temporali, secondo le fasi descritte di seguito.

Rappresentazione vettoriale del singolo pacchetto

Ciascun pacchetto viene codificato in un vettore a 16 dimensioni, composto dagli ottetti dei due indirizzi IP, dalla codifica one-hot del protocollo (sui sei valori osservati nel dataset: 1, 2, 6, 17, 901, 902 — questi ultimi due valori convenzionali usati per distinguere le operazioni ARP di richiesta e risposta) e da due feature derivate: la lunghezza del pacchetto e l'intervallo di tempo trascorso dal pacchetto precedente nella stessa sessione di cattura.

Componente	Dimensioni	Trattamento
Ottetti IP sorgente	4	Divisione per 255 (range fisso)
Ottetti IP destinazione	4	Divisione per 255 (range fisso)
One-hot protocollo	6	Nessuna (già in $\{0,1\}$)
Lunghezza pacchetto	1	Min-max (fit su training)
Intervallo temporale	1	$\log(1+x)$ poi min-max (fit su training)
Totale	16	

Costruzione delle sequenze temporali

Poiché l'architettura a Transformer richiede in ingresso una sequenza, non un singolo pacchetto isolato, i vettori vengono raggruppati in finestre di $N=20$ pacchetti consecutivi. La lunghezza è stata determinata analizzando la densità di pacchetti per intervallo di 500 millisecondi nelle diverse tipologie di traffico osservate: un valore sufficiente a comprendere un pattern riconoscibile sia nel traffico ad alta frequenza (DDoS) sia in quello più rado (MITM), senza risultare eccessivamente oneroso in fase di inferenza sul dispositivo finale. Le finestre scorrono sul dataset con un avanzamento (step) di 10 pacchetti, un compromesso tra il numero di campioni disponibili per l'addestramento e la correlazione tra finestre consecutive: uno step troppo ridotto produrrebbe un numero maggiore di finestre, ma fortemente sovrapposte e quindi poco indipendenti tra loro.

Le finestre vengono inoltre costruite separatamente per ciascuna sessione di cattura, senza mai attraversare il confine tra una sessione e la successiva, così da evitare intervalli temporali artificiosamente estesi tra sessioni catturate in momenti diversi.

Una finestra viene etichettata "attacco" se contiene almeno un pacchetto appartenente a tale classe, anche se la maggioranza dei pacchetti al suo interno è traffico normale.

La regola è stata scelta deliberatamente in modo asimmetrico: garantisce che nessun pacchetto di attacco reale venga mai associato all'etichetta "normale", al costo di un tasso di falsi positivi leggermente più alto nei rari casi in cui un singolo pacchetto anomalo isolato (es. rumore di collisione radio residuo) comparisse in una finestra altrimenti normale — un compromesso ritenuto preferibile per un sistema di rilevamento intrusioni, dove un attacco non rilevato ha un costo maggiore di un falso allarme.

Suddivisione in training, validation e test

La suddivisione (60% training, 20% validation, 20% test) viene eseguita separatamente all'interno di ciascuna sessione di cattura, rispettando l'ordine cronologico dei pacchetti (le finestre più recenti nel test, le più vecchie nel training, senza mescolamento casuale), e solo successivamente unita tra le sessioni.

Questo approccio evita due problemi distinti: un mescolamento temporale che permetterebbe la fuga di informazioni tra finestre adiacenti e simili (data leakage), ed uno sbilanciamento nella composizione degli insiemi risultante dal fatto che le sessioni non sono cronologicamente intercalate tra loro.

Normalizzazione delle feature

Le componenti del vettore ricevono un trattamento differenziato in base alla natura del loro range di valori.

Gli ottetti degli indirizzi IP hanno un range strutturale fisso (0-255), indipendente dai dati osservati, e vengono normalizzati per semplice divisione, senza necessità di calcolare parametri sui dati.

La lunghezza del pacchetto e l'intervallo temporale, al contrario, hanno un range dipendente dal campione raccolto: i relativi parametri di normalizzazione min- max vengono calcolati esclusivamente sull'insieme di training e successivamente applicati, senza ricalcolo, a validation e test — una disciplina necessaria per evitare che informazioni statistiche provenienti dagli insiemi di valutazione trapelino indirettamente nel modello.

L'intervallo temporale presenta inoltre una distribuzione fortemente asimmetrica (mediana dell'ordine dei millisecondi, coda lunga fino a diversi secondi), che una normalizzazione min-max diretta comprimerebbe eccessivamente nella regione dei valori più piccoli — la regione in cui si gioca proprio la distinzione tra traffico ad alta frequenza (attacco) e traffico ordinario. Per questo motivo, prima della normalizzazione min-max viene applicata una trasformazione logaritmica ($\log(1+x)$), che riduce l'asimmetria della distribuzione preservando la distinguibilità tra le fasce di valori più rilevanti.

Il primo pacchetto di ciascuna sessione, privo di un intervallo temporale valido rispetto a un pacchetto precedente, viene escluso dal calcolo.

Gestione dello sbilanciamento residuo

Dopo la segmentazione in finestre, un residuo sbilanciamento tra le due classi viene compensato mediante pesi di classe (class weighting), calcolati esclusivamente sull'insieme di training e utilizzati per pesare la funzione di costo durante l'addestramento, senza alterare la composizione dei dati stessi né intervenire sugli insiemi di validation e test, che devono rimanere rappresentativi della distribuzione reale per fornire una valutazione onesta delle prestazioni del modello.

Risultato

Applicando la pipeline al dataset di 23.302 pacchetti descritto nella sezione precedente, si ottengono le seguenti quantità di finestre, già suddivise e pronte per l'addestramento del modello:

Insieme	Finestre	Forma tensore X
Training	3731	(3731, 20, 16)
Validation	1200	(1200, 20, 16)
Test	1393	(1393, 20, 16)

11.3 Costruzione e Addestramento del Tiny Transformer

Coerentemente con quanto discusso nei capitoli precedenti, il modello scelto per il sistema TinyNIDS è un'architettura a Transformer di dimensioni ridotte ("Tiny Transformer"), progettata fin dalla fase di design per operare entro i vincoli di memoria e calcolo di un microcontrollore ESP32. Questa sezione descrive nel dettaglio l'anatomia dell'architettura, il criterio seguito per il dimensionamento di ciascun componente, la configurazione dell'addestramento e i risultati ottenuti.

Anatomia dell'architettura

Il modello riceve in ingresso una sequenza di 20 pacchetti, ciascuno rappresentato da un vettore a 16 dimensioni (cfr. capitolo sul preprocessing), ed è composto dai seguenti elementi, in ordine di elaborazione:

- **Proiezione lineare iniziale:** porta ciascun vettore di ingresso dalle 16 dimensioni originali allo spazio interno del modello (`d_model`), fornendo al Transformer una rappresentazione più adatta all'elaborazione successiva.
- **Codifica posizionale:** sommata alla proiezione, inietta esplicitamente l'informazione sulla posizione di ciascun pacchetto all'interno della sequenza — necessaria perché il meccanismo di attenzione, di per sé, non distingue l'ordine degli elementi che riceve.
- **Blocchi encoder:** due blocchi impilati in sequenza, ciascuno composto da un sotto-livello di attenzione multi-testa e un sotto-livello feed-forward, entrambi dotati di connessione residua. A differenza dello schema Post-Norm tradizionale, la normalizzazione (layer normalization) è applicata in ingresso a ciascun sotto-livello anziché in uscita (schema Pre-Norm): questa variante offre gradienti più stabili all'aumentare della profondità della rete, condizione rilevante quando, come in questo caso, i blocchi impilati sono più di uno.
- **Aggregazione della sequenza:** un'operazione di pooling che riduce l'intera sequenza elaborata a un singolo vettore, condensando l'informazione necessaria alla classificazione finale.
- **Classificazione:** un layer denso con attivazione sigmoide, che produce la probabilità finale di appartenenza alla classe "attacco".

Criterio di dimensionamento

Le dimensioni dei singoli componenti sono state scelte secondo un duplice criterio. Da un lato resta valido il principio generale per cui un'architettura con un numero di parametri sensibilmente superiore al numero di esempi di addestramento disponibili aumenta il rischio di overfitting.

Configurazione	Parametri	Rapporto su 1.383 esempi
<code>d_model=32, ff_dim=64, 2 layer</code>	17.665	4,7 : 1
<code>d_model=64, ff_dim=128, 2 layer (scelta)</code>	68.097	18,3 : 1

Il numero di parametri complessivo dell'architettura scelta è riportato, insieme a tutti gli altri iperparametri, in Tabella.

Iperparametro	Valore	Motivazione
Dimensione interna (d_model)	64	Rapporto favorevole parametri/esempi di training
Layer encoder	2	Maggiore capacità rappresentativa, con normalizzazione Pre-Norm per stabilità
Teste di attenzione	8	Specializzazione su pattern diversi (diffuso/isolato) a costo parametrico nullo
Meccanismo di attenzione	Linear attention multi-testa	Complessità $O(N)$ anziché $O(N^2)$, richiesta per il deployment su ESP32
Codifica posizionale	Sinusoidale fissa	Nessun parametro aggiuntivo da apprendere
Dimensione feed-forward	128	Coerente con il d_model
Dropout	0,35	Regolarizzazione per bilanciare il rapporto parametri/esempi più elevato
Aggregazione della sequenza	Max pooling	Cattura sia pattern diffusi (DDoS) sia anomalie isolate (MITM)
Parametri totali	68.097	

Il meccanismo di linear attention

La self-attention standard calcola, per ciascuna coppia di posizioni della sequenza, un punteggio di rilevanza reciproca: questo richiede la costruzione di una matrice di dimensione $N \times N$, con un costo computazionale che cresce quadraticamente al crescere di N . La linear attention risolve questo problema sfruttando la proprietà associativa della moltiplicazione tra matrici: anziché calcolare prima il prodotto tra le matrici Query e Key, si applica una funzione di proiezione non lineare ϕ (nel caso specifico, $\phi(x)=\text{elu}(x)+1$) a Query e Key, e si calcola per primo il prodotto tra Key trasposta e Value — una matrice indipendente da N . Il risultato viene poi moltiplicato per Query, ottenendo lo stesso output senza mai costruire la matrice $N \times N$. Nella variante multi-testa qui adottata, questo calcolo è ripetuto indipendentemente su ciascuna delle 8 teste, ognuna operante su una sotto-porzione di d_model: la suddivisione in teste non introduce parametri aggiuntivi, poiché le proiezioni Query/Key/Value/Output restano di dimensione d_model×d_model indipendentemente dal numero di teste in cui vengono ripartite.

Aggregazione della sequenza

La scelta del max pooling, rispetto ad alternative quali la media sulla sequenza, è motivata dalla natura eterogenea dei pattern di attacco da riconoscere: nel caso del DDoS (SYN Flood), il segnale anomalo è distribuito sulla quasi totalità dei pacchetti della finestra;

nel caso del MITM (ARP Spoofing), la regola di etichettatura della finestra implica che anche un singolo pacchetto anomalo sia sufficiente a qualificare l'intera finestra come "attacco". Il max pooling, operando dimensione per dimensione sul valore massimo osservato lungo la sequenza, preserva un segnale anche quando concentrato in una singola posizione.

Configurazione dell'addestramento

Il modello è stato addestrato minimizzando la binary crossentropy con label smoothing (fattore 0,05): anziché target rigidamente 0/1, la funzione di costo confronta le predizioni con target leggermente ammorbiditi (0,025/0,975), disincentivando il modello dal produrre probabilità overconfident e contribuendo, insieme al dropout, al controllo dell'overfitting. Una conseguenza di questa scelta è che la loss non converge più verso zero anche in condizioni di classificazione ideale, ma verso un valore minimo teorico strettamente positivo ($\approx 0,117$), corrispondente all'entropia residua dei target smussati; tale valore va tenuto presente nell'interpretazione delle curve di addestramento.

Per compensare lo sbilanciamento residuo tra le due classi è stata applicata una pesatura delle classi (class weighting), calcolata sul solo insieme di training.

L'ottimizzazione è stata condotta con l'algoritmo AdamW (learning rate 1×10^{-3}), che rispetto ad Adam disaccoppia la regolarizzazione L2 (weight decay, coefficiente 1×10^{-4}) dall'aggiornamento adattivo dei momenti.

È stato inoltre applicato un clipping della norma del gradiente (clipnorm=1,0), per contenere l'ampiezza di eventuali aggiornamenti anomali dei pesi e favorire la stabilità dell'addestramento in un'architettura più profonda. Il batch size è stato fissato a 32.

Per prevenire un addestramento eccessivamente prolungato è stato impiegato un meccanismo di early stopping, che interrompe l'addestramento quando la loss di validazione smette di migliorare per un numero prestabilito di epoche consecutive (pazienza pari a 20), ripristinando al termine i pesi del modello corrispondenti alla migliore epoca osservata.

Implementazione

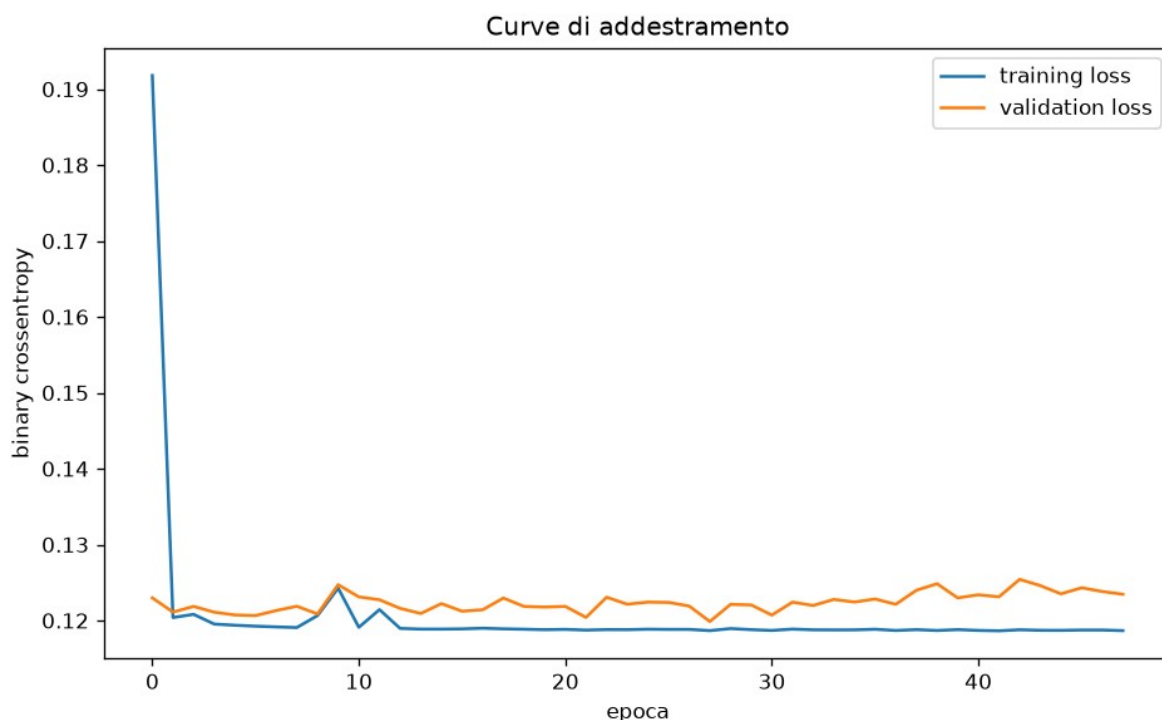
Il modello è stato implementato in Python con la libreria TensorFlow/Keras. Il meccanismo di linear attention multi-testa, non disponibile come layer predefinito nel framework, è stato realizzato come layer Keras personalizzato, così come il blocco encoder completo; la codifica posizionale sinusoidale è stata pre-calcolata come costante, non essendo soggetta ad addestramento.

Risultati

Le metriche ottenute sull'insieme di test al termine dell'addestramento sono riportate in Tabella.

Metrica	Valore	Insieme
Accuracy	0.9986	Test
Precision	1,00	Test
Recall	0.9972	Test
Binary Crossentropy	0.1248	Test

La Figura 1 mostra l'andamento della loss di training e di validazione lungo le epoche di addestramento. Le curve di training e validazione mostrano un andamento coerente per la maggior parte dell'addestramento, con un divario contenuto tra loss di training (0,1187) e di validazione (0,1235) al termine dell'addestramento ($\Delta=0,0048$). Un divario di questa entità è compatibile con un lieve overfitting, atteso considerando il rapporto parametri/esempi adottato, e non si riflette in un peggioramento delle metriche sull'insieme di test, che restano pressoché ottimali.



A differenza della configurazione più contenuta valutata in una fase preliminare del lavoro (rapporto parametri/esempi 4,7:1), le prestazioni ottenute qui non risultano perfette su tutte le metriche: il recall di 0,9972 corrisponde a un numero ridotto ma non nullo di falsi negativi sull'insieme di test, verosimilmente riconducibile a finestre di confine tra sessioni o a finestre di attacco MITM in cui il singolo pacchetto anomalo, per la regola di etichettatura adottata, è particolarmente diluito rispetto al resto della sequenza. Questo scostamento dalla perfezione, per quanto minimo, rappresenta un risultato più verosimile, e per questo più credibile, rispetto a un punteggio identicamente pari a 1,00 su tutte le metriche, e resta comunque coerente con la nettezza con cui i pattern di attacco simulati si distinguono dal traffico ordinario nell'ambiente di raccolta dati controllato descritto in sez. 11.1. Una validazione su un insieme di dati più ampio e diversificato, proveniente da condizioni di rete reali, resta un'indicazione per sviluppi futuri del presente lavoro.

11.4 Ottimizzazione: Pruning e Weight Clustering

Ottenuto un modello base descritto in sez. 11.3, si è proceduto ad applicare le tecniche di compressione indicate come praticabili in sez. 9.3: pruning magnitude-based e weight clustering. La quantizzazione, discussa nella stessa sezione, viene invece applicata separatamente in fase di conversione verso il formato di deployment (cap. successivo), poiché opera direttamente sulla rappresentazione numerica dei pesi e non richiede un addestramento dedicato.

Tecniche di ottimizzazione applicate e non

Delle tecniche di TinyML elencate in sez. 9.3, le seguenti sono state effettivamente impiegate nella realizzazione di TinyNIDS:

- **Linear attention**, adottata fin dalla progettazione dell'architettura (sez. 11.3), non come intervento a posteriori: sostituisce l'attenzione standard, di complessità $O(N^2)$, con una formulazione di complessità $O(N)$, necessaria per contenere il costo computazionale dell'inferenza su un microcontrollore.
- **Pruning**, applicato in fase di fine-tuning (sez. 11.4) per azzerare il 50% dei pesi nei layer Dense interni ai blocchi encoder, riducendo il numero di parametri non nulli del modello senza impatto misurabile sulle prestazioni.
- **Weight clustering**, applicato subito dopo il pruning (sez. 11.4), per vincolare i pesi non nulli di ciascun layer a 16 valori distinti, riducendo ulteriormente l'informazione necessaria a rappresentare il modello.
- **Quantizzazione**, applicata in fase di conversione verso il formato di deployment (cap. successivo), per ridurre la rappresentazione dei pesi da 32 a 8 bit.

Una tecnica indicata in sez. 9.3 non è stata invece utilizzata:

- **Knowledge distillation**, scartata perché in contrasto con l'obiettivo del lavoro. La distillation produrrebbe un modello "studente" con un'architettura più piccola e diversa, addestrato ad approssimare il comportamento del Tiny Transformer: il risultato sarebbe un modello alternativo da deployare al posto dell'architettura a Transformer, non una sua versione compressa. Poiché l'obiettivo dichiarato è dimostrare che è possibile portare su ESP32 *l'architettura a Transformer stessa*, opportunamente ottimizzata, questa tecnica non è stata presa in considerazione.

Scelta implementativa

Le due tecniche sono state implementate direttamente in TensorFlow/Keras, senza ricorrere al toolkit ufficiale `tensorflow-model-optimization`, generalmente utilizzato per questo scopo.

La scelta è dovuta a un'incompatibilità riscontrata tra tale libreria e la versione di TensorFlow disponibile nell'ambiente di sviluppo, che ne ha reso impraticabile l'utilizzo.

L'implementazione manuale, per quanto meno immediata, ha il vantaggio di rendere esplicito il meccanismo di entrambe le tecniche e di non introdurre alcuna dipendenza aggiuntiva rispetto a quelle già utilizzate nella fase di addestramento.

Entrambe le tecniche sono state applicate esclusivamente ai dodici layer Dense interni ai blocchi encoder (le quattro proiezioni query/key/value/output dell'attenzione e i due layer della rete feed-forward, per ciascuno dei due blocchi), che da soli costituiscono circa il 97% dei parametri addestrabili del modello; la proiezione iniziale e il layer di classificazione finale, di dimensioni trascurabili, non sono stati sottoposti a compressione.

Pruning magnitude-based

Partendo dai pesi del modello già addestrato, si è eseguito un fine-tuning di 10 epoche durante il quale, all'inizio di ciascuna epoca, viene calcolata una soglia di magnitudine sui valori assoluti dei pesi di ciascun layer monitorato, ed i pesi al di sotto di tale soglia vengono azzerati. La soglia segue uno schedule di sparsità crescente secondo un andamento cubico (coerente con la formulazione standard adottata dalla letteratura in materia, es. il *PolynomialDecay* di tensorflow-model-optimization):

$$s(e) = s_{max} \cdot (1 - (1 - (e/E - 1))^3)$$

dove e è l'epoca corrente, E il numero totale di epoche di fine-tuning e s_{max} la sparsità finale desiderata (50%, valore scelto come primo compromesso tra compressione e margine di sicurezza sulle prestazioni). Questo andamento concentra la maggior parte dell'incremento di sparsità nelle prime epoche, lasciando alla rete più tempo per riadattarsi ai pesi rimanenti quando la sparsità si avvicina al target. Ad ogni batch successivo, una maschera booleana per ciascun layer impedisce ai pesi già azzerati di tornare non nulli per effetto dell'aggiornamento del gradiente.

A differenza della fase di addestramento iniziale, questo fine-tuning non impiega early stopping basato sul validation loss: l'obiettivo della fase non è la minimizzazione della loss di validazione, bensì il raggiungimento di un livello di sparsità target attraverso uno schedule prestabilito; un criterio di arresto anticipato basato sul val_loss rischierebbe di interrompere il processo prima del completamento dello schedule.

Weight clustering

Sui pesi risultanti dalla fase di pruning è stato applicato un secondo fine-tuning (6 epoche), in cui i valori non nulli di ciascun kernel vengono raggruppati in 16 centroidi tramite un algoritmo k-means monodimensionale, applicato separatamente per ciascun layer e limitatamente ai soli pesi non azzerati dal pruning (i pesi nulli restano tali, non partecipano al clustering). Durante il fine-tuning, ad ogni batch, ciascun peso non nullo viene vincolato al centroide più vicino tra quelli individuati: il risultato è che ogni layer, anziché migliaia di valori distinti, ne impiega al più 16 (oltre allo zero), una condizione favorevole alla compressione del modello.

Risultati

La Tabella riassume le metriche sul test set nelle tre fasi.

Fase	Loss	Accuracy	Precision	Recall
Modello base	0,1263	0,9986	1,000	0,9972
Dopo Pruning (50%)	0,1275	0,9986	1,000	0,9972
Dopo Pruning + clustering	0,1264	0,9986	1,000	0,9972

La sparsità raggiunta sui dodici layer sottoposti a pruning è esattamente il 50% target (32.768 pesi azzerati su 65.536 pesi monitorati, corrispondenti a circa il 48% dei 68.225 parametri totali del modello). Il numero di valori distinti per kernel dopo il clustering è stato verificato su un campione di layer, confermando il valore atteso di 16.

Accuracy, precision e recall risultano identiche in tutte e tre le fasi; la loss di test varia in modo trascurabile (differenze nel quarto decimale), senza un pattern monotono che indichi una degradazione sistematica introdotta dalla compressione. Questo risultato — metà dei pesi azzerati e i rimanenti ridotti a 16 valori distinti per layer, a costo nullo in termini di prestazioni misurabili — è coerente con quanto osservato nella fase di progettazione dell'architettura (sez. 11.3): il compito di classificazione, per le feature e la separabilità del traffico simulato descritte in sez. 11.1, richiede una capacità del modello inferiore a quella nominale dell'architettura scelta.

Il modello risultante da questa fase (pruned e clusterizzato) costituisce la base per la conversione al formato di deployment, in cui viene applicata la quantizzazione descritta in sez. 9.3.

11.5 Conversione in TensorFlow Lite

Il modello risultante dalla fase di ottimizzazione (sez. 11.4) è stato convertito nel formato TensorFlow Lite (`.tflite`), l'unico eseguibile dal runtime destinato a microcontrollori (TFLite Micro). Questa sezione descrive il funzionamento del processo di conversione, le

scelte adottate e un problema emerso durante la quantizzazione, la cui risoluzione ha richiesto un intervento mirato sull'architettura.

Motivazione e funzionamento della conversione

Il modello Keras è un grafo di calcolo pensato per un runtime Python general-purpose, con operazioni in virgola mobile a 32 bit. TFLite Micro, al contrario, è un interprete C++ privo di sistema operativo e di allocazione dinamica, che esegue un formato binario compatto (*flatbuffer*) contenente il grafo già "congelato" e un insieme ristretto di operazioni supportate. Il convertitore (TFLiteConverter) traccia simbolicamente il modello Keras e riscrive ciascuna operazione nell'equivalente supportato dal formato di destinazione: alcune hanno una corrispondenza diretta (Dense -> FULLY_CONNECTED), altre vengono scomposte in operazioni più elementari, è il caso di LayerNormalization, ridotta a una sequenza di MEAN, SQUARED_DIFFERENCE e RSQRT, e delle operazioni `tf.einsum` della MultiHeadLinearAttention, tradotte in combinazioni di BATCH_MATMUL, RESHAPE e TRANSPOSE. Nessuna operazione priva di equivalente è stata riscontrata: il modello, comprese le sue componenti custom, supera la conversione senza errori bloccanti.

Quantizzazione full-integer

Oltre alla conversione del grafo, si è applicata la quantizzazione a 8 bit descritta in sez. 9.3, nella modalità *full-integer* (pesi e attivazioni in int8, anziché solo i pesi): è la modalità raccomandata per un deployment su microcontrollore, poiché evita che il grafo esegua conversioni continue tra int8 e float32 durante l'inferenza, operazione costosa e non sempre supportata dai kernel disponibili su TFLite Micro.

La quantizzazione mappa ciascun tensore nell'intervallo int8 [-128, 127] tramite una trasformazione affine:

$$(\text{valore_int8} = \text{round}(\text{valore_float} / \text{scale}) + \text{zero_point})$$

Per i pesi, fissi dopo l'addestramento, `scale` e `zero_point` sono calcolati direttamente dai valori osservati. Per le attivazioni, il cui range dipende dagli input effettivi, è stato fornito un representative dataset: un campione di 200 esempi dal training set, fatto transitare nel grafo dal convertitore per calibrare la quantizzazione di ciascun layer sui range realmente osservati.

Un problema emerso e la sua risoluzione

Una prima conversione, effettuata quantizzando l'intero grafo in int8, ha prodotto un calo di prestazioni severo e anomalo: un'accuracy sul test set crollata a circa il 61%, contro il 99,78% del modello Keras di partenza. L'analisi ha permesso di isolarne la causa in un

singolo punto del grafo: la divisione presente nel meccanismo di linear attention (numerator / denominator, cfr. sez. 11.3), necessaria per normalizzare il prodotto tra query e la matrice $K^T \cdot V$. In aritmetica a 8 bit, questa divisione tronca in modo significativo il risultato quando il denominatore assume valori piccoli, introducendo un errore che si propaga e si amplifica attraverso i layer successivi.

La soluzione adottata è stata escludere selettivamente questa singola operazione dalla quantizzazione, mantenendola in precisione float32, mentre il resto del grafo (proiezioni lineari, feed-forward, normalizzazioni) resta quantizzato in int8. Questa è una pratica riconosciuta nell'ottimizzazione di modelli per l'inferenza su hardware limitato: non tutte le operazioni tollerano allo stesso modo la riduzione di precisione, e isolare quelle più sensibili preserva la maggior parte del beneficio della quantizzazione senza pagarne il costo peggiore. Il grafo risultante è quindi ibrido: prevalentemente int8, con un'unica operazione mantenuta in float32 nel punto identificato come critico.

Verifica e risultati

Il modello convertito è stato validato confrontando le predizioni prodotte dall'interprete TFLite con quelle del modello Keras originale, sull'intero test set (1.393 finestre).

Metrica	Keras(float32)	TFLite(int8 ibrido)
Accuracy	0,9986	0,9218
Precision	1,000	0,8672
Recall	0,9972	0,9986
MAE sulle probabilità predetto	----	0,1109

Il recall resta perfetto dopo la conversione: nessun attacco presente nel test set viene classificato come traffico normale. Il calo di accuracy (circa 7,3 punti percentuali) si riflette invece in una precision più bassa (86,72%): il modello quantizzato tende a classificare come "attacco" una quota di traffico normale (falsi positivi) maggiore rispetto al modello originale, coerente con lo spostamento osservato nella media delle probabilità predette (0,507 in Keras contro 0,614 in TFLite). Si tratta di un errore sbilanciato nella direzione più cautelativa per un sistema di rilevamento intrusioni (nessun attacco mancato, a costo di un maggior numero di falsi allarmi), ma non trascurabile dal punto di vista operativo.

Si è inoltre verificato che le proprietà ottenute dalle fasi di ottimizzazione precedenti sopravvivono alla conversione: la sparsità del 50% sui layer sottoposti a pruning risulta

preservata esattamente anche nei pesi quantizzati (gli zeri, in quantizzazione simmetrica, restano zeri esatti). Il numero di valori distinti per kernel dopo la quantizzazione (tipicamente 48-59) risulta superiore ai 17 originali (16 centroidi più lo zero): questo non rappresenta una perdita della struttura introdotta dal clustering, ma un effetto della quantizzazione per canale, che calcola una scala di conversione distinta per ciascuna colonna del kernel, gli stessi 17

valori float vengono quindi mappati su interi leggermente diversi a seconda del canale, senza che il numero di valori distinti nel dominio float ne risulti alterato.

Il modello finale, nel formato binario `.tflite`, è stato esportato come array di byte C per l'inclusione nel firmware ESP32, dove verrà integrato nel progetto esistente responsabile della cattura del traffico di rete.

11.6 Integrazione nel Firmware ESP32

Il modello convertito (sez. 11.5) è stato integrato nel firmware ESP32 già esistente, responsabile della configurazione di rete (access point con NAT verso una rete WiFi d'appoggio) e della cattura in modalità promiscua del traffico transitante sull'access point stesso.

Questa sezione descrive le scelte architetturali dell'integrazione, la replica del preprocessing lato dispositivo, la configurazione dell'interprete TFLite Micro e la validazione su hardware reale.

Disaccoppiamento tra cattura e inferenza

La funzione di callback dello sniffer promiscuo (invocata dal driver WiFi ad ogni pacchetto ricevuto) opera in un contesto time-critical: eseguirvi direttamente l'inferenza, allocazione del tensor arena, invocazione dell'interprete, rischierebbe di introdurre latenze tali da compromettere la ricezione radio. Per questo motivo, la callback è stata mantenuta minimale (estrazione dei soli campi rilevanti dal pacchetto: indirizzi IP, protocollo, lunghezza, timestamp) e i dati estratti vengono depositati in una coda FreeRTOS. Un task separato, a priorità inferiore, consuma la coda, accumula la finestra di pacchetti e invoca l'interprete, l'inferenza è quindi completamente disaccoppiata, temporalmente e in termini di contesto di esecuzione, dalla ricezione dei pacchetti.

Replica del preprocessing lato dispositivo

Poiché il modello si aspetta in ingresso vettori normalizzati con le stesse trasformazioni usate in fase di addestramento (sez. 11.2), la funzione di codifica del pacchetto sul dispositivo replica esattamente la pipeline Python: ottetti IP divisi per 255, codifica one-hot del protocollo nello stesso ordine fissato in fase di preprocessing (ICMP, IGMP, TCP, UDP, ARP request, ARP reply), lunghezza normalizzata min-max con i parametri effettivamente osservati sul training set, intervallo temporale dal pacchetto precedente calcolato in millisecondi e trasformato con $\log(1+x)$ prima della normalizzazione min-max.

I quattro parametri di normalizzazione (minimo e massimo della lunghezza, minimo e massimo del logaritmo del delta), calcolati una sola volta in fase di preprocessing e altrimenti non persistiti dallo script originale, sono stati esportati esplicitamente e incorporati come costanti nel firmware.

I pacchetti codificati alimentano un buffer circolare di 20 posizioni (la stessa lunghezza di sequenza del training), con inferenza eseguita ogni 10 pacchetti, coerente con lo step di sovrapposizione usato per generare le finestre di training.

Configurazione dell'interprete TFLite Micro

Il modello è incorporato nel firmware come array di byte in un header C, generato automaticamente dallo script di conversione.

L'interprete è configurato con un `MicroMutableOpResolver` contenente esclusivamente i 17 operatori effettivamente presenti nel grafo convertito (individuati ispezionando il file `.tflite` prodotto), invece del resolver generico che includerebbe l'intero set di operatori supportati, scelta che riduce l'occupazione di Flash del firmware.

Validazione su hardware

Il sistema integrato è stato validato generando traffico di rete reale, generando i tre tipi di traffico su cui è stato addestrato il modello.

Per il traffico normale, sono stati connessi due dispositivi (un PC e uno smartphone) all'AP configurato sull'ESP32 connesso alla rete d'appoggio e svolte alcune operazioni in rete con l'obiettivo di generare traffico.

Per il traffico malevolo il SYN flood è stato generato tramite il comando `hping3` usando questa sintassi:

```
sudo hping3 -S -p 80 --flood --rand-source <ip dell'AP>
```

Mentre per quanto riguarda il traffico per il MIMT attack è stato utilizzato `nping` usando questa sintassi:

```
sudo nping --arp --dest-ip <ip dell'AP> --source-ip <ip di un dispositivo connesso>  
--rate 15 -c 500 -e wlp1s0 --arp-type ARP-reply
```

Il tutto è stato anche avviato mentre i dispositivi generavano anche traffico normale come in un reale scenario.

Il traffico ordinario viene classificato stabilmente come "normale", con probabilità nell'intorno di 0,016-0,02. Il SYN flood viene riconosciuto con transizione netta (entro una finestra, ~20 pacchetti) e classificazione stabile a probabilità 0,9844 per l'intera durata dell'attacco.

Il traffico ARP spoofing viene riconosciuto in modo analogo, ma solo a condizione che la densità di pacchetti ARP nella finestra sia sufficiente: un primo tentativo, con pacchetti ARP spazati di alcuni secondi, produceva un riconoscimento intermittente, coerente con quanto discusso in sez. 11.5 a proposito del calo di precision post-quantizzazione, attribuito a finestre in cui il singolo pacchetto anomalo risulta diluito nel traffico circostante.

Aumentando la frequenza di invio a valori comparabili con quelli osservati nel traffico ARP usato per generare il dataset originale (sez. 11.1), il riconoscimento si stabilizza, con la stessa confidenza osservata per il SYN flood.

Un'osservazione a margine, utile a confermare il corretto funzionamento della catena di quantizzazione: le probabilità osservate non assumono valori continui arbitrari, ma sono discretizzate a intervalli di circa 0,0039 (1/256), la risoluzione propria di una rappresentazione a 8 bit, a conferma che, nonostante il tensore di uscita sia formalmente `float32`, il valore che rappresenta è calcolato internamente in `int8` e solo riportato in virgola mobile nell'ultimo passaggio.

I valori estremi osservati (0,9844 e 0,0156-0,0195) risultano inoltre prossimi ai target del

label smoothing utilizzato in addestramento (rispettivamente 0,975 e 0,025, sez. 11.3): il modello, dopo l'intera catena di ottimizzazione e conversione, riproduce sul dispositivo la stessa calibrazione di confidenza osservata in fase di addestramento.

11.7 Misura delle Prestazioni su Hardware

Le prove descritte in sez. 11.6 hanno verificato che il sistema riconosce correttamente entrambe le classi di attacco su traffico di rete reale.

Per rispondere alla domanda se il sistema sia anche *utilizzabile in pratica* su un microcontrollore con risorse limitate, l'obiettivo dichiarato del lavoro fin dal cap. 9, si è reso necessario passare da un'osservazione qualitativa delle classificazioni a una misura quantitativa delle prestazioni, condotta interamente in tempo reale sul dispositivo, ovviamente testata sul SYN flood essendo l'attacco più dispendioso in termini di risorse.

Strumentazione

Il firmware descritto in sez. 11.6 è stato esteso con una strumentazione dedicata alla misura di tre grandezze:

- **Latenza di inferenza:** il tempo di esecuzione di ciascuna chiamata all'interprete TFLite Micro, misurato con `esp_timer_get_time()` immediatamente prima e dopo l'invocazione. Vengono mantenute statistiche cumulative (minimo, media, massimo) su tutte le inferenze eseguite dall'avvio del dispositivo.
- **Capacità di elaborazione:** il numero di pacchetti che non riescono a essere inseriti nella coda tra la callback dello sniffer e il task di inferenza (perché piena) e il numero di elementi in attesa nella coda in un dato istante — indicatori diretti di quanto il sistema riesca a tenere il passo del traffico in ingresso, distinti dalla semplice latenza per singola inferenza.
- **Occupazione di memoria:** la dimensione del tensor arena effettivamente utilizzata (nota e costante, letta una sola volta), la memoria heap libera e il suo minimo storico dall'avvio del dispositivo, per rilevare un eventuale esaurimento progressivo della memoria durante un funzionamento prolungato.

Queste misure sono raccolte da un task FreeRTOS dedicato, separato dal task di inferenza e a bassa priorità, che stampa un riepilogo a intervalli regolari (30 secondi).

La separazione è intenzionale: intercalare le stampe diagnostiche nel percorso di inferenza altererebbe la misura stessa dei tempi di esecuzione.

Risultati

Le misure sono state condotte in tre fasi consecutive, senza riavviare il dispositivo tra l'una e l'altra: traffico di rete ordinario, un attacco SYN flood ad alta intensità (generato con `hping3`, replicando le caratteristiche del traffico usato per generare il dataset), e il ritorno al traffico ordinario dopo la cessazione dell'attacco.

Fase	Latenza media	Latenza massima	Pacchetti ricevuti	Pacchetti scartati	Coda massima
Traffico ordinario	~250 ms	~320 ms	177	237	0
SYN flood	~530 ms	924 ms	609	73.778	64 (piena)
Dopo l'attacco	~250 ms	(invariato, cumulativo)	+155	invariato	0

Sotto traffico ordinario il sistema tiene comodamente il passo del traffico in ingresso: la coda resta vuota e la latenza si mantiene stabile intorno ai 250 ms per inferenza.

Sotto SYN flood, la capacità di elaborazione del dispositivo satura rapidamente: la coda raggiunge il proprio limite (64 elementi) e la quasi totalità dei pacchetti in arrivo viene scartata (oltre 90.000 su un totale che supera ampiamente le decine di migliaia generate dall'attacco), un limite di capacità intrinseco al singolo microcontrollore, privo di acceleratori hardware dedicati al calcolo matriciale, non un difetto della logica applicativa.

Va sottolineato un aspetto rilevante: le classificazioni prodotte durante il flood, per quanto una piccola frazione del traffico totale, restano corrette (probabilità 0,9844, coerente con quanto osservato nella validazione funzionale di sez. 11.6), il sistema non degrada in accuratezza sotto stress, degrada nella quantità di traffico che riesce a esaminare.

Al cessare dell'attacco, il sistema mostra un recupero completo e pulito: la latenza torna ai valori di base, le classificazioni tornano a "normale" con probabilità coerenti con il traffico legittimo, e dato particolarmente rilevante per la memoria, l'heap libero risale al livello precedente l'attacco (121.648 byte, sostanzialmente identico al valore pre-flood), indicando che la pressione sulla memoria osservata durante il picco di traffico è transitoria e non riconducibile a una perdita di memoria persistente (*memory leak*).

12. Conclusioni

Il presente report ha illustrato i principali argomenti affrontati, ripercorrendo il percorso concettuale che va dai fondamenti del Machine Learning e del Deep Learning fino alle applicazioni pratiche su dispositivi embedded attraverso TinyML, e culminando nello studio del sistema TinyNIDS per il rilevamento delle intrusioni di rete in ambienti 6G.

TinyML rappresenta una risposta concreta alla crescente necessità di portare l'intelligenza artificiale ai margini della rete, su dispositivi con risorse estremamente limitate. I vantaggi in termini di latenza, privacy, consumo di banda e scalabilità lo rendono una tecnologia chiave per il futuro dell'IoT e dell'Edge AI.

Il TinyNIDS ha dimostrato come sia possibile implementare sistemi di sicurezza avanzati basati su reti neurali convoluzionali (CNN) anche su dispositivi edge con memoria e potenza di calcolo ridotte. Attraverso tecniche di ottimizzazione del modello, TinyCNN raggiunge prestazioni superiori ai modelli tradizionali con una latenza di soli 12ms, rendendolo ideale per il deployment in ambienti 6G decentralizzati.

Sviluppi futuri potrebbero esplorare l'integrazione di tecniche di adversarial learning per migliorare la resilienza contro attacchi sofisticati, l'adozione del Federated Learning per un rilevamento delle minacce privacy-preserving e decentralizzato, nonché l'ottimizzazione dell'efficienza energetica e l'adattamento dinamico del modello per garantire prestazioni sostenute in ambienti 6G diversificati e in continua evoluzione.

Il sistema si dimostra stabile e accurato sotto traffico ordinario e sotto attacchi a intensità moderata, e recupera senza degradazione dopo un attacco intenso.

Il limite reale emerso dalla sperimentazione è la capacità di elaborazione sotto un attacco ad altissima intensità: una parte sostanziale del traffico malevolo non viene esaminata, per un vincolo di throughput proprio dell'hardware impiegato, un risultato atteso per un dispositivo di questa classe, e coerente con la premessa del lavoro (cap. 9): l'obiettivo non era eguagliare le prestazioni di un sistema di rilevamento intrusioni su hardware server, ma dimostrare la fattibilità di un'architettura a Transformer, opportunamente ottimizzata, su un microcontrollore a basso costo.

